

Ejercicio 1 - Orquestador Concurrente

Código

```
import asyncio
import time

async def clasificar_ticket(ticket_id: int) -> str:
    print(f"{time.strftime('%H:%M:%S')} Ticket {ticket_id}: inicia clasificación")
    await asyncio.sleep(0.4)
    return "categoría: acceso a cuenta"

async def resumir_ticket(ticket_id: int) -> str:
    print(f"{time.strftime('%H:%M:%S')} Ticket {ticket_id}: inicia resumen")
    await asyncio.sleep(0.7)
    return "resumen: el cliente no puede ingresar"

async def evaluar_urgencia(ticket_id: int) -> str:
    print(f"{time.strftime('%H:%M:%S')} Ticket {ticket_id}: inicia evaluación de urgencia")
    espera = 2.3 if ticket_id in {3, 8} else 0.9
    await asyncio.sleep(espera)
    return "urgencia: media"

async def procesar_ticket(ticket_id: int, semaforo: asyncio.Semaphore) -> dict:
    async with semaforo:
        print(f"{time.strftime('%H:%M:%S')} Ticket {ticket_id}: simulación iniciada")
        try:
            async with asyncio.timeout(2.0):
                clasificacion, resumen, urgencia = await asyncio.gather(
                    clasificar_ticket(ticket_id),
                    resumir_ticket(ticket_id),
                    evaluar_urgencia(ticket_id),
                )
            return {
                "ticket": ticket_id,
                "estado": "completado",
                "clasificacion": clasificacion,
                "resumen": resumen,
                "urgencia": urgencia,
            }
        except TimeoutError:
            print(f"{time.strftime('%H:%M:%S')} Ticket {ticket_id}: excedió el timeout de 2 segundos")
            return {"ticket": ticket_id, "estado": "timeout"}

async def main() -> None:
    semaforo = asyncio.Semaphore(2)
    tareas = [procesar_ticket(ticket_id, semaforo) for ticket_id in range(1, 11)]
    resultados = await asyncio.gather(*tareas)

    print("\nResultados finales")
    for resultado in resultados:
        print(resultado)

if __name__ == "__main__":
    asyncio.run(main())
```

Explicación breve

El programa simula el análisis de tickets de soporte. Para cada ticket, clasificación, resumen y urgencia se inician en paralelo con **asyncio.gather**.

Cada simulación tiene un límite total de dos segundos mediante **asyncio.timeout**. Si una de las tareas tarda demasiado, se captura **TimeoutError** y el ticket queda marcado como timeout, sin frenar los demás.

El **Semaphore(2)** permite procesar solamente dos tickets a la vez. Aunque se crean diez tareas, el resto espera su turno; así se evita sobrecargar un servicio externo.